Multi-byte Regex Functions

The mbstring module also provides a set of functions for handling regular expressions, with support for multi-byte encodings (as you might expect). The core preg_* functions we've just been looking at will support UTF-8 data – provided a u modifier is added after the closing delimiter of the pattern – but are otherwise not multi-byte aware and certainly do not support encodings such as UTF-16, etc. – for non-UTF-8 encodings, there are the mb_ereg_* functions.

Internally, the mbstring regex functions use the open source Oniguruma regular expression library https://github.com/kkos/oniguruma, which supports a wide variety of character encodings and expression syntaxes. Detailed documentation for the Oniguruma regex syntax can be found here: https://github.com/kkos/oniguruma/blob/master/doc/RE, though it is very similar to PCRE, everything demonstrated so far in this section should work just fine. One important practical difference, though, is that mb_ereg_* functions have a slightly different pattern syntax – they do not use delimiters.

```
$squid = "\u{1F991}";
var_dump(mb_ereg('.', $squid, $matches));
print_r($matches);
```

```
Output:
bool(true)
Array
(
    [0] => 🦑
)
```

Our pattern is a simple dot (note the lack of delimiters), which matches any single character, so mb_ereg() returns true. The UTF-8 encoding for U+1f991 (Squid) is *0xF0 0x9F 0xA6 0x91*, and it's clear that the function is correctly decoding the bytes to a single codepoint, as we can see in the $matches array. Now if we try the same string and pattern with preg_match(), the outcome depends on the modifiers used:

```
preg_match('/./', $squid, $matches);
print_r($matches);
```